

Resumen Ejecutivo de Avance

Proyecto de Estandarización de Instrumentación y Control — Ingenio La Florida

Fecha: 1 de septiembre de 2026 **Informe anterior:** Resumen_Ejecutivo_Avance_310826 (31/08/2026)

1. Resumen del período

Jornada de **consolidación de la Tags App**: se le sumó una capa de ingeniería (visualización de lazos, validación ISA-5.1 y precarga de componentes faltantes) y una capa de productividad y UX (exportación a Excel, selección múltiple, traductor a lectura humana, usuario obligatorio, refresh automático y botón de recarga), más el saneamiento puntual de la base y la corrección de dos bugs de persistencia/refresh.

Nueve frentes:

- 1 **Visualización de lazos de control** (`generar_esquema_lazo`) con grafo dirigido NetworkX + Matplotlib embebido en Tkinter.
- 2 **Validador ISA-5.1** (`validador_isa.py`): detecta los componentes obligatorios de un lazo cerrado y los indicadores locales.
- 3 **Precarga automática** de los componentes faltantes del lazo desde el formulario (botón de confirmación).
- 4 **Exportación a Excel** de los tags seleccionados en el Paso 5.
- 5 **Selección múltiple** del Treeview (Ctrl/Shift) con rangos por teclado.
- 6 **Traductor de tags a lectura humana** en el Paso 2, en tiempo real.
- 7 **Mantenimiento de base**: `eliminar_tags_masivos()` (13 tags eliminados).
- 8 **UX/refresh**: usuario obligatorio, refresh automático del Paso 5 y botón "■ Recargar".
- 9 **Bugs corregidos**: el Paso 5 ya no se vacía al seleccionar, y se verificó la persistencia completa (incluidos comentarios) en las actualizaciones.

Dependencias nuevas instaladas en el entorno: `networkx 3.6.1`, `matplotlib 3.11.1`, `openpyxl 3.1.5`.

2. Visualización de lazos de control

2.1 `generar_esquema_lazo(lazo_id)`

Se agregó una función a la Tags App que grafica el lazo de control pedido:

- **Consulta**: extrae los instrumentos del mismo lazo (ej. 200_PT_002, 200_PIC_002, 200_PV_002) con el número de lazo y área indicados, excluyendo Retirado.
- **Grafo dirigido**: nodos = tags, aristas = lógica Sensor → Controlador → Actuador (clasificación por letra de función: T sensor, C/IC controlador, V actuador; si no hay controlador, se enlaza Sensor → Actuador).
- **Layout**: `networkx.spring_layout()`.
- **Render**: ventana `Toplevel` con `FigureCanvasTkAgg` (Matplotlib embebido), nodos coloreados por rol (verde sensor, azul controlador, rojo actuador) y título con el conteo.

2.2 `probar_lazo.py` (utilidad)

Programa de prueba para abrir el esquema de un lazo desde la terminal:

```
py probar_lazo.py          # abre el lazo 300_001 por defecto
py probar_lazo.py 200_002  # abre el lazo indicado (formato area_numero)
```

3. Validador ISA-5.1

Nuevo módulo app_etiquetas/validador_isa.py:

Componente	Descripción
REGLAS_VALIDACION	Diccionario con LAZOS_CERRADOS (SENSOR=T, CONTROLADOR=C, ACTUADOR=V) e INDICADORES_LOCALES (I, R)
auditar_tag_recien_guardado(tag_guardado, lista_tags_existentes)	Extrae función (ej. V) y lazo (ej. 035) del tag, cruza contra las reglas y devuelve (faltantes_obligatorios, sugerencias_locales)

Integrado en on_guardar (post-INSERT): al guardar un tag, si al lazo le faltan componentes obligatorios (transmisor/controlador/actuador) se notifica, y si no hay indicador local se sugiere uno.

4. Precarga automática de componentes faltantes del lazo

El flujo de validación dejó de ser solo informativo:

- Se reemplazó el showinfo por messagebox.askyesno("Asistente ISA-5.1", "¿Desea precargar el siguiente componente obligatorio del lazo [tag]?").
- Si el usuario acepta, se llama a precargar_tag_en_formulario(tag_sugerido), que:
- setea el campo editable entry_tag con el tag sugerido (delete + insert),
- selecciona automáticamente Área/Variable/Función en los combobox correspondientes,
- regenera la propuesta y hace entry_tag.focus_set().
- Si hay varios faltantes, se precarga el **primero** de faltantes_obligatorios (extraído con _extraer_tag_faltante).

Para soportar esto se agregó el campo editable entry_tag en el Paso 1, sincronizado con la propuesta (_set_entry_tag), y on_guardar ahora guarda el valor del campo.

5. Exportación a Excel (Paso 5)

- Nuevo botón "**Exportar Excel**" (verde #2ECC71, texto blanco, negrita) en el Paso 5.
- on_exportar_excel(): toma los tags seleccionados del Treeview (tree.selection()), consulta la base con todas las columnas (SELECT t.*, a.codigo, a.nombre ...) y vuelca a exports/tags_{fecha}.xlsx con **openpyxl**.
- Muestra messagebox.showinfo("Exportación completada", "X tags exportados").

Se instaló openpyxl y el botón de exportación quedó en el Paso 5 (se retiró el botón "Exportar CSV" previo del Paso 2).

6. Selección múltiple y traductor humano

6.1 Selección múltiple del Treeview

- `selectmode='extended'` (Ctrl+Click y Shift+Click nativos).
- Bindings explícitos `<Shift-Up>/<Shift-Down>` → `_extender_seleccion("up"/"down")` para extender la selección por rangos con el teclado.
- `<Button-1>` → `focus_set()` para asegurar el foco al hacer clic.

6.2 Traductor de tags (lectura humana)

- `traducir_tag_humano(tag_completo, diccionarios)`: separa área, función, número, mapea el código de área al nombre (ej. 200 → Destilería), la primera letra de la función a la variable (ej. L → Nivel) y el resto a la función detalle (ej. T → Transmisor), y devuelve "{funcion_detalle} de {variable}, lazo {numero}, área {area_code} ({nombre_area})". Formato inválido → "Formato de tag no estándar".
- Diccionarios de mapeo (MAPEO_AREAS, MAPEO_VARIABLES, MAPEO_FUNCIONES, agrupados en DICIONARIOS) con ejemplos básicos.
- `ttk.Label` de traducción en el Paso 2, debajo de "Tag propuesto", actualizado en tiempo real al cambiar la propuesta o el campo editable.

7. Mantenimiento de base: eliminación masiva de tags

Nueva función `eliminar_tags_masivos(lista_tags)` en `database.py` (junto a `eliminar_tag`):

- Borra primero las filas de auditoria (FK sin ON DELETE CASCADE) y luego las de tags, con `conn.commit()`.
- Devuelve la cantidad de tags efectivamente eliminados (los inexistentes se ignoran).

Se ejecutó sobre una lista de 13 tags de Destilería/Calderas (instrumentos de los lazos 001-002, visores LG, válvulas y un transmisor), **13 eliminados** y ninguno restante en la base.

8. UX y refresh

- **Usuario obligatorio:** `on_guardar()` valida que "Registrado por" no esté vacío antes del INSERT; si lo está → `messagebox.showwarning("Usuario requerido", "Debe ingresar un usuario antes de guardar")`.
- **Refresh automático del Paso 5:** nueva función `refrescar_paso5()` que limpia el Treeview (`delete(*get_children())`), re-consulta la base, reinserta todas las filas y actualiza el contador "X tag(s) en total". Se llama tras INSERT (`on_guardar`), UPDATE (`on_actualizar`), DELETE (`on_eliminar`) y al arrancar.
- **Botón "■ Recargar":** junto a "Exportar Excel", llama a `_recargar_app()`: reinicia `db.init_db()`, recarga catálogos (áreas/variables/funciones/usuarios), limpia formulario y Treeviews, y recarga la grilla desde la base.

9. Bugs corregidos

Bug	Corrección
El Paso 5 (Treeview) se vaciaba al seleccionar un tag del Paso 2	on_seleccionar_existente ya no filtra/limpia la grilla; ahora llama a refrescar_paso5() y el Treeview permanece poblado
Persistencia de campos modificados	Verificado: on_actualizar() pasa todos los campos (incluido comentarios) a db.actualizar_tag(), cuyo UPDATE incluye comentarios = ? y hace conn.commit(). No faltaba ningún campo

10. Archivos modificados hoy

Archivo	Cambio
app_etiquetas/app_tags.py	generar_esquema_lazo, entry_tag + precargar_tag_en_formulario + _extraer_tag_faltante + _set_entry_tag, on_exportar_excel, _extender_seleccion, refrescar_paso5, _recargar_app, traducir_tag_humano + DICCIONARIOS + lbl_traduccion, usuario obligatorio, refresh automático, botón "■ Recargar" y "Exportar Excel", correcciones de selección y persistencia
app_etiquetas/validador_isa.py	Nuevo — REGLAS_VALIDACION y auditar_tag_recien_guardado
app_etiquetas/database.py	eliminar_tags_masivos(lista_tags)
probar_lazo.py	Nuevo — utilidad para abrir el esquema de un lazo
app_etiquetas/tags_ingenio.db	Saneamiento: 13 tags eliminados
Dependencias	Instalados networkx, matplotlib, openpyxl